

SupplyChainX — Blockchain Part

Complete practical guide for your internship blockchain module: problem, solution, roles, smart contract, Remix, product hash, QR workflow, Ganache, backend/frontend integration, testing and 2-day plan.

1. Your Blockchain Responsibility

The proposal assigns the Blockchain Team: product authenticity verification, tamper-proof product history, smart contracts (optional), ownership transfer records and the supply-chain audit trail. The proposal's example journey is Manufacturer → Distributor → Warehouse → Retailer → Customer.

Manufacturer → Distributor → Warehouse → Retailer → Customer
↓
Blockchain product history

2. Problem → Solution

Problems: counterfeit products, lack of transparency, data tampering and difficulty tracing defective products.

Solution: register a genuine product on an Ethereum-compatible blockchain, store a product hash and trusted lifecycle events, record ownership transfers and allow customers to verify the product through a QR-linked verification page.

3. Role-Based Model

Role	Allowed actions
Manufacturer	Register product; transfer; update location; view history
Distributor	Receive/transfer; update location; view history
Warehouse	Receive/transfer; update location; view history
Retailer	Receive/transfer; update location/status; view history
Customer	Scan QR / enter ID; verify authenticity; view journey

4. File Structure

```
blockchain/  
  ■■■ contracts/  
  ■ ■■■ SupplyChain.sol      ← main contract  
  ■ ■■■ ISupplyChain.sol    ← optional interface  
  ■■■ scripts/  
  ■ ■■■ deploy.js  
  ■■■ test/  
  ■ ■■■ SupplyChain.test.js  
  ■■■ README.md
```

Later backend:
backend/src/blockchain/
 ■■■ contract.js
 ■■■ abi/SupplyChain.json

For the 2-day prototype, SupplyChain.sol is the priority. The interface can remain optional until backend integration.

5. Smart Contract Data

```
enum Role {
    None, Manufacturer, Distributor,
    Warehouse, Retailer, Customer
}

struct Product {
    string productId;
    bytes32 productHash;
    address manufacturer;
    address currentOwner;
    string currentLocation;
    string status;
    uint256 createdAt;
    bool exists;
}

struct History {
    address from;
    address to;
    string location;
    string action;
    uint256 timestamp;
}
```

Mappings:

```
mapping(string => Product) private products;
mapping(string => History[]) private productHistory;
mapping(address => Role) public roles;
```

6. Main Functions

```
setRole()           → assign role
registerProduct()    → manufacturer creates product
transferProduct()    → current owner transfers product
updateLocation()     → current owner updates location/status
verifyProduct()      → compare supplied hash with stored hash
getProduct()         → read current product
getProductHistory()  → read full journey
```

7. Role Permissions in Solidity

```
modifier onlySupplyChainParticipant() {
    Role role = roles[msg.sender];

    require(
        role == Role.Manufacturer ||
        role == Role.Distributor ||
        role == Role.Warehouse ||
        role == Role.Retailer,
        "Not an authorized supply chain participant"
    );
    _;
}
```

Use onlyManufacturer for setRole() and registerProduct(). Use onlySupplyChainParticipant for transferProduct() and updateLocation(). Customer is restricted to verification/read operations.

8. setRole() Values

```
0 = None
1 = Manufacturer
2 = Distributor
3 = Warehouse
4 = Retailer
5 = Customer
```

The deploying account becomes the first Manufacturer. The Manufacturer assigns roles to other addresses.

9. Remix Test Sequence

1. Compile SupplyChain.sol with Solidity ^0.8.20.
2. Deploy from Account 0.
3. Account 0 becomes Manufacturer.
4. From Account 0:
 setRole(Account1, 2)
 setRole(Account2, 3)
 setRole(Account3, 4)
 setRole(Account4, 5)
5. Register SCX-001 from Account 0.
6. Transfer Account 0 → Account 1.
7. Transfer Account 1 → Account 2.
8. Transfer Account 2 → Account 3.
9. Read getProductHistory().
10. Test verifyProduct().

10. First Product Test

[illegible]

The fixed hash above is only for the first Remix test. In the final application, the backend should calculate a deterministic hash from agreed product data.

11. Product Lifecycle

Register:
Manufacturer → Blockchain

Transfer:
Manufacturer → Distributor
Distributor → Warehouse
Warehouse → Retailer

History:
Product Registered
→ Ownership Transferred
→ Ownership Transferred
→ Ownership Transferred

12. QR Generation

QR generation is an application/backend responsibility, not a Solidity function. After product registration, the application can create a QR containing a verification URL such as:

<https://supplychainx.com/verify/SCX-001>

A QR library converts that text into the visible square QR image. The same QR stays on the physical product while its blockchain history changes.

13. Customer QR Verification

```
Customer
↓
Scan QR
↓
/verify/SCX-001
↓
React verification page
↓
GET /api/products/SCX-001
↓
Backend
↓
Blockchain
↓
verifyProduct()
getProduct()
getProductHistory()
↓
✓ Authentic + product journey
```

14. Backend ↔ Blockchain

```
React
  ■ HTTP/REST
  ▼
Backend
  ■ ethers.js
  ▼
Ethereum RPC
  ▼
Ganache
  ▼
SupplyChain.sol
```

The backend needs the deployed contract address, ABI and RPC URL. Write operations also need a signing account. Keep private keys in environment variables.

```
BLOCKCHAIN_RPC_URL=...
CONTRACT_ADDRESS=...
PRIVATE_KEY=...
```

15. Backend API Mapping

Method	Endpoint	Contract call
POST	/api/products	registerProduct()
POST	/api/products/:id/transfer	transferProduct()
PUT	/api/products/:id/location	updateLocation()
GET	/api/products/:id	getProduct()
GET	/api/products/:id/history	getProductHistory()
GET	/api/products/:id/verify	verifyProduct()

16. Backend Contract Connection

```
const provider =
  new ethers.JsonRpcProvider(process.env.BLOCKCHAIN_RPC_URL);

const wallet = new ethers.Wallet(
  process.env.PRIVATE_KEY, provider
);

const contract = new ethers.Contract(
  process.env.CONTRACT_ADDRESS,
  ABI,
  wallet
);
```

17. Frontend Role-Based UI

Manufacturer: Dashboard → Register Product → Generate/View QR → Transfer → History.

Distributor: Received Products → Transfer → Location → History.

Warehouse: Incoming Products → Location → Transfer → History.

Retailer: Received Products → Status/Sale → History.

Customer: Scan QR or enter Product ID → Verification → Authenticity + Journey.

18. Customer Verification Result

✓ AUTHENTIC PRODUCT

Product: SCX-001
Manufacturer: ABC Electronics
Current Owner: XYZ Retail
Current Location: Guwahati Warehouse
Status: In Transit

SUPPLY CHAIN JOURNEY

✓ Manufacturer
↓
✓ Distributor
↓
✓ Warehouse
↓
✓ Retailer

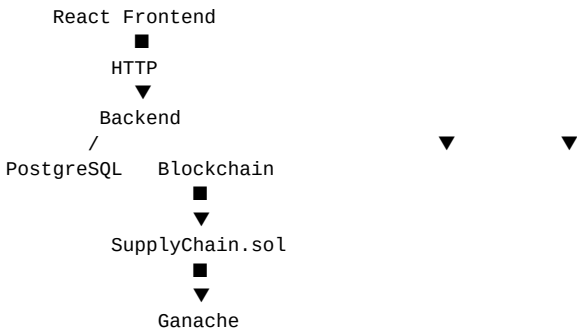
19. Ganache

Ganache is the local Ethereum-compatible development blockchain listed in the project's proposed technology stack. Remix VM can be used first for contract testing; Ganache becomes useful when the backend connects through an RPC endpoint.

Stage 1:
Remix VM → SupplyChain.sol

Stage 2:
Backend → ethers.js → Ganache RPC → SupplyChain.sol

20. Final Architecture



Blockchain = trusted product/history layer
Backend = application + blockchain bridge
Frontend = role UI + QR scanning

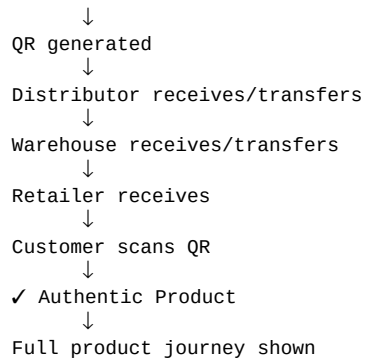
21. 2-Day Plan

Day 1: Solidity → Remix compile/deploy → roles → register → transfer → location → history → verify → permission tests.

Day 2: Backend connection → APIs → QR generation → frontend verification → end-to-end test → demo preparation.

22. Demo Flow

Manufacturer logs in
↓
Register SCX-001



23. Important Scope Rule

Do not add tokens, NFTs, cryptocurrency payments, complex DAO logic, or unnecessary extra contracts. The proposal's blockchain purpose is traceability and authenticity. Focus on a working, demonstrable prototype.